

A PRACTICAL GUIDE

Multithreading in C# From First Principles to Production Patterns

Threads, the Thread Pool, Tasks, async/await, Parallel & Concurrent Programming — explained from the ground up, with working code examples.

A hands-on companion for learning modern C# concurrency

Preface

This book teaches multithreading, parallel and concurrent programming, and the `async / await` model in C#, starting from zero and building up to patterns you'll actually use in production code.

Every concept is introduced with:

1. **The problem** — why the concept exists.
2. **The concept** — explained simply, with a real-world analogy where useful.
3. **A working code example** — small, runnable, and focused on one idea.
4. **Common pitfalls** — mistakes almost everyone makes at first.

You should be comfortable with basic C# syntax (classes, methods, delegates/lambda). Nothing else is assumed.

Table of Contents

1. Why Multithreading? Processes, Threads, and Tasks
2. Your First Thread
3. The Trouble With Shared State: Race Conditions
4. Locking and Synchronization Primitives
5. The Thread Pool
6. Tasks: The Modern Way to Run Work
7. Async and Await
8. Combining Tasks: Continuations, WhenAll, WhenAny
9. Cancellation
10. Parallel Programming: Parallel.For and PLINQ
11. Concurrent Collections
12. Producer-Consumer Pattern (Putting It All Together)
13. Common Pitfalls and Best Practices
14. Cheat Sheet / Quick Reference

Chapter 1 — Why Multithreading? Processes, Threads, and Tasks

1.1 The problem

A computer program that does one thing at a time is simple to reason about, but slow whenever it has to wait — for a disk read, a network reply, a sensor over a serial port, or just a long calculation. If your program only ever does one thing at a time, the whole application freezes while it waits, even though the CPU is sitting idle.

Multithreading lets a program do more than one thing "at the same time" (either truly in parallel on multiple CPU cores, or interleaved on a single core), so that waiting on one thing doesn't block everything else.

1.2 Processes vs. Threads

- A **process** is a running program with its own memory space. Two processes cannot see each other's variables directly.
- A **thread** is a unit of execution *inside* a process. All threads in a process share the same memory (the same objects, the same static fields). A process always has at least one thread (the "main thread").
- **Multiple threads sharing memory** is what makes multithreading powerful (fast communication) and dangerous (two threads can corrupt shared data if they're not careful — see Chapter 3).

```
Process ("MyApp.exe")
├─ Thread 1 (main thread: runs Main())
├─ Thread 2 (background: polling a sensor)
└─ Thread 3 (background: writing to a log file)
```

1.3 Concurrency vs. Parallelism

These two words are often confused:

- **Concurrency** means *structuring* a program so multiple tasks can make progress without waiting for each other to finish — they may or may not run at the exact

same instant. A single CPU core can be "concurrent" by rapidly switching between tasks.

- **Parallelism** means multiple tasks *literally* running at the same instant, which requires multiple CPU cores.

Modern C# gives you tools for both:

Goal	Tool
Don't block while waiting for I/O (disk, network, serial port)	<code>async</code> / <code>await</code>
Run CPU-heavy work on a background thread so the UI/main thread stays responsive	<code>Task.Run</code> , <code>Thread</code>
Split a big CPU-bound job across all cores to finish faster	<code>Parallel.For</code> , <code>PLINQ</code>
Coordinate multiple workers safely	locks, concurrent collections

1.4 Threads vs. Tasks vs. `async/await` — the 30-second map

C# gives you three related but distinct abstractions, and beginners often mix them up:

- **Thread** — the raw operating-system thread. Low-level, expensive to create (roughly 1 MB of stack per thread), rarely used directly in modern code.
- **Task** — a higher-level "unit of work that will complete in the future," usually run on the **thread pool** (a pool of reusable threads) instead of a dedicated OS thread. This is the modern building block for concurrency in C#.
- **`async / await`** — a *language feature* built on top of `Task` that lets you write asynchronous, non-blocking code (especially I/O-bound code) that reads like ordinary sequential code.

This book covers them in that order, because each one is built on the concept before it.

Chapter 2 — Your First Thread

2.1 Creating a thread

The `System.Threading.Thread` class is the most primitive way to run code on a separate thread. You rarely use it directly in modern applications (Tasks are almost always better — see Chapter 6), but understanding it builds the right mental model.

```
using System;
using System.Threading;

class Program
{
    static void Main()
    {
        // Create a thread that will run the PrintNumbers method.
        Thread worker = new Thread(PrintNumbers);
        worker.Start(); // The new thread starts running here.

        // Meanwhile, the main thread keeps going immediately.
        for (int i = 0; i < 5; i++)
        {
            Console.WriteLine($"Main thread: {i}");
        }

        worker.Join(); // Wait for the worker thread to finish before exiting.
        Console.WriteLine("Both threads are done.");
    }

    static void PrintNumbers()
    {
        for (int i = 0; i < 5; i++)
        {
            Console.WriteLine($"Worker thread: {i}");
        }
    }
}
```

What's happening:

- `new Thread(PrintNumbers)` creates a thread object, but does *not* start it yet.
- `.Start()` actually launches it — from that point, both the main thread and the worker thread run concurrently.
- The output order of "Main thread: x" and "Worker thread: x" lines is **not deterministic** — the OS scheduler decides. Run this program twice and you may

see different interleavings. This unpredictability is the essence of concurrent programming.

- `.Join()` blocks the calling thread until the target thread finishes. Without it, `Main()` could exit before the worker thread is done.

2.2 Passing data to a thread

```
using System;
using System.Threading;

class Program
{
    static void Main()
    {
        // ParameterizedThreadStart accepts a single object argument.
        Thread worker = new Thread(DoWork);
        worker.Start("sensor-42");
        worker.Join();

        // A lambda is usually cleaner, and lets you capture multiple variables.
        int deviceId = 7;
        Thread worker2 = new Thread(() => DoWorkTyped(deviceId, "COM3"));
        worker2.Start();
        worker2.Join();
    }

    static void DoWork(object? data)
    {
        Console.WriteLine($"Processing device: {data}");
    }

    static void DoWorkTyped(int deviceId, string port)
    {
        Console.WriteLine($"Device {deviceId} on port {port}");
    }
}
```

2.3 Background vs. foreground threads

By default, threads you create are **foreground threads**: the process will not exit until they finish. Set `IsBackground = true` to make a thread a **background thread**, which the runtime will kill automatically when the process exits (useful for things like a never-ending polling loop that shouldn't keep your app alive).

```
Thread poller = new Thread(PollSensorForever) { IsBackground = true };
poller.Start();
// The process can exit even if PollSensorForever() never returns.
```

2.4 Why you should rarely use `Thread` directly

- Each `Thread` reserves about 1 MB of stack space and costs real time to create/destroy — expensive if you need hundreds of short-lived operations.
- There's no built-in way to get a *return value* or an *exception* back from a `Thread` — you have to build that yourself.
- The **thread pool** (Chapter 5) and **Tasks** (Chapter 6) solve both problems and are almost always the better choice today. `Thread` is still useful when you need a long-lived, dedicated thread (e.g., a continuously-running background worker that talks to hardware) or fine control over thread priority/stack size.

Chapter 3 — The Trouble With Shared State: Race Conditions

3.1 The problem

Because threads in the same process share memory, two threads can read and write the same variable at the same time. This causes a **race condition**: the final result depends on the unpredictable timing ("race") of the threads, rather than on the logic of the program.

```
using System;
using System.Threading;

class Program
{
    static int counter = 0;

    static void Main()
    {
        Thread t1 = new Thread(Increment);
        Thread t2 = new Thread(Increment);
        t1.Start();
        t2.Start();
        t1.Join();
        t2.Join();

        // You would EXPECT 2,000,000. You will usually get something LESS.
        Console.WriteLine($"Final counter: {counter}");
    }

    static void Increment()
    {
        for (int i = 0; i < 1_000_000; i++)
        {
            counter++; // NOT ATOMIC - this is the bug.
        }
    }
}
```

3.2 Why `counter++` is not safe

`counter++` looks like one operation, but the CPU actually performs three steps:

1. Read `counter` into a register.

2. Add 1 to it.
3. Write it back to `counter`.

If Thread A reads the value (say, 5), and before it writes back 6, Thread B also reads 5 and writes 6, **one increment is lost** — both threads thought they were incrementing from 5, but only one increment actually "stuck." Multiply this by millions of iterations across multiple threads and the final count comes out lower than expected, by an unpredictable amount that can even change between runs.

This is the single most important concept in concurrent programming: **any time two threads can read-modify-write the same piece of shared, mutable state without coordination, you have a bug waiting to happen** — even if it doesn't show up every time you run the program.

3.3 The fix: mutual exclusion

The fix is to ensure only one thread can execute the "read-modify-write" sequence at a time. C#'s simplest tool for this is the `lock` statement, covered in the next chapter.

```
static int counter = 0;
static readonly object counterLock = new object();

static void Increment()
{
    for (int i = 0; i < 1_000_000; i++)
    {
        lock (counterLock)
        {
            counter++; // Now safe: only one thread can be inside the lock at a time.
        }
    }
}
```

With the `lock` in place, running the earlier program now reliably prints `2000000` every time.

Chapter 4 — Locking and Synchronization Primitives

4.1 `lock` (the go-to tool)

`lock (obj) { ... }` is shorthand for using `Monitor.Enter/Monitor.Exit` with a `try/finally`. Only one thread at a time can be inside a block locked on the same object.

```
private readonly object _sync = new object();
private readonly List<string> _log = new List<string>();

public void AddLogEntry(string entry)
{
    lock (_sync)
    {
        _log.Add(entry); // List<T> is NOT thread-safe on its own – must be protected.
    }
}
```

Rules of thumb for `lock`:

- Always lock on a **private, dedicated, readonly object** (like `_sync` above) — never on `this`, a string, or a publicly accessible object, because outside code could accidentally lock on the same object and cause a deadlock.
- Keep the code inside a `lock` block **short**. Never do I/O, call unknown code, or `await` inside a lock.
- You cannot `await` inside a `lock` block — the compiler will not allow it (see Chapter 7 for why).

4.2 Deadlocks

A **deadlock** happens when two threads each hold a lock the other one needs, and neither can proceed:

```

// Thread A
lock (lockA)
{
    Thread.Sleep(100);
    lock (lockB) { /* ... */ }
}

// Thread B
lock (lockB)
{
    Thread.Sleep(100);
    lock (lockA) { /* ... */ } // <-- deadlock!
}

```

Thread A holds `lockA` and waits for `lockB`. Thread B holds `lockB` and waits for `lockA`. Neither ever finishes.

Fix: always acquire multiple locks in the same, consistent order everywhere in your code (e.g., always `lockA` before `lockB`).

4.3 `Monitor.TryEnter` — locking with a timeout

```

if (Monitor.TryEnter(_sync, TimeSpan.FromSeconds(1)))
{
    try
    {
        // Got the lock within 1 second.
    }
    finally
    {
        Monitor.Exit(_sync);
    }
}
else
{
    Console.WriteLine("Could not acquire lock in time – avoiding a potential deadlock.");
}

```

4.4 `Interlocked` — lock-free atomic operations

For simple operations like incrementing a counter, `System.Threading.Interlocked` is faster than a full `lock` because it uses special CPU instructions instead of an operating-system lock.

```

using System.Threading;

private static int _counter = 0;

static void Increment()
{
    for (int i = 0; i < 1_000_000; i++)
    {
        Interlocked.Increment(ref _counter); // Atomic: no race condition possible.
    }
}

// Other useful Interlocked operations:
Interlocked.Add(ref _counter, 5);
int old = Interlocked.Exchange(ref _counter, 0); // Set to 0, return the previous value.
Interlocked.CompareExchange(ref _counter, 10, 0); // If _counter == 0, set it to 10.

```

4.5 SemaphoreSlim — limiting concurrent access to N threads

A `lock` allows exactly one thread in at a time. A `SemaphoreSlim` allows up to N threads in at a time — useful for throttling (e.g., "no more than 3 simultaneous connections to this device").

```

using System.Threading;
using System.Threading.Tasks;

// Allow at most 3 concurrent operations.
private static readonly SemaphoreSlim _semaphore = new SemaphoreSlim(3, 3);

static async Task AccessResourceAsync(int id)
{
    await _semaphore.WaitAsync(); // Blocks (asynchronously) if 3 are already inside.
    try
    {
        Console.WriteLine($"Task {id} entered.");
        await Task.Delay(1000); // Simulate work.
    }
    finally
    {
        _semaphore.Release(); // ALWAYS release, even on exceptions – hence try/finally.
    }
}

```

`SemaphoreSlim` is important because, unlike `lock`, it supports `WaitAsync()` — meaning it's safe to use with `async / await` code (Chapter 7).

4.6 ReaderWriterLockSlim — many readers, one writer

When data is read far more often than it's written, a plain `lock` is wasteful — it blocks readers from each other even though concurrent reads are perfectly safe.

```
private static readonly ReaderWriterLockSlim _rwLock = new ReaderWriterLockSlim();
private static Dictionary<int, string> _cache = new Dictionary<int, string>();

static string? Read(int key)
{
    _rwLock.EnterReadLock();
    try
    {
        return _cache.TryGetValue(key, out var value) ? value : null;
    }
    finally
    {
        _rwLock.ExitReadLock();
    }
}

static void Write(int key, string value)
{
    _rwLock.EnterWriteLock();
    try
    {
        _cache[key] = value;
    }
    finally
    {
        _rwLock.ExitWriteLock();
    }
}
```

Multiple threads can hold the read lock simultaneously; the write lock is exclusive of everything.

Chapter 5 — The Thread Pool

5.1 The problem with creating a new `Thread` per job

If your program needs to do 1,000 short pieces of work, creating 1,000 `Thread` objects would be extremely wasteful: each reserves ~1 MB of stack and takes real time to create and tear down.

5.2 The solution: a pool of reusable threads

The **.NET thread pool** keeps a set of already-created threads around and hands them out to run short pieces of work, reusing them afterward instead of destroying them. This avoids the overhead of constant thread creation/destruction.

```
using System;
using System.Threading;

class Program
{
    static void Main()
    {
        for (int i = 0; i < 10; i++)
        {
            int captured = i; // Capture the loop variable (see note below).
            ThreadPool.QueueUserWorkItem(state =>
            {
                Console.WriteLine($"Job {captured} running on thread
{Thread.CurrentThread.ManagedThreadId}");
            });
        }

        Thread.Sleep(500); // Give the pool time to run everything before Main exits.
    }
}
```

Note on capturing loop variables: in modern C# (C# 5+), `foreach` variables are captured correctly per-iteration, but with a plain `for` loop using `i`, you still generally want to copy it into a new local variable (`captured`) before using it in a closure, to make the intent explicit and avoid the classic "closure captures the loop variable" bug in older or unusual code paths.

5.3 You will rarely call `ThreadPool.QueueUserWorkItem` directly

In modern C#, you virtually never call `ThreadPool.QueueUserWorkItem` directly. Instead, you use `Task.Run(...)` (Chapter 6), which queues work to the *same* thread pool but gives you:

- A return value (`Task<T>`).
- Exception propagation (exceptions inside the task are captured, not silently lost).
- Composability (you can chain, wait on, and combine tasks).

Think of the thread pool as the "engine room" running underneath both `ThreadPool.QueueUserWorkItem` and `Task.Run` — `Task` is simply the modern, safer API built on top of it.

Chapter 6 — Tasks: The Modern Way to Run Work

6.1 What is a `Task`?

A `System.Threading.Tasks.Task` represents "some work that will complete at some point in the future" — possibly with a result. Think of it as a **promise**: you get the `Task` object back immediately, and you can check on it, wait for it, or chain more work onto it, all without blocking right away.

- `Task` — represents an operation with **no return value** (like a `void` method, but trackable).
- `Task<TResult>` — represents an operation that **produces a value** of type `TResult` when it completes.

6.2 Starting a task with `Task.Run`

```
using System;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        Console.WriteLine("Starting long calculation...");

        // Task.Run queues the work onto the thread pool and returns immediately.
        Task<long> task = Task.Run(() => SumOfSquares(50_000_000));

        Console.WriteLine("This prints immediately, while the sum is still being
computed.");

        long result = await task; // Wait for it to finish and get the result.
        Console.WriteLine($"Result: {result}");
    }

    static long SumOfSquares(int upTo)
    {
        long sum = 0;
        for (int i = 0; i < upTo; i++)
        {
            sum += (long)i * i;
        }
        return sum;
    }
}
```

`Task.Run` is the right tool when you have **CPU-bound** work (a calculation) that you want to push onto a background thread so it doesn't block the caller.

6.3 Getting results and handling exceptions

```
static async Task Main()
{
    Task<int> task = Task.Run(() =>
    {
        throw new InvalidOperationException("Something went wrong in the background.");
#pragma warning disable CS0162
        return 42;
#pragma warning restore CS0162
    });

    try
    {
        int result = await task; // The exception is re-thrown here, at the await point.
    }
    catch (InvalidOperationException ex)
    {
        Console.WriteLine($"Caught: {ex.Message}");
    }
}
```

This is a key advantage over raw `Thread`: an unhandled exception on a plain `Thread` crashes the process; an exception inside a `Task` is captured and only re-thrown when you `await` (or `.Wait()` / `.Result`) it.

6.4 `Task.Delay` VS. `Thread.Sleep`

```
// BAD in async code: blocks the entire thread for 2 seconds, wasting it.
Thread.Sleep(2000);

// GOOD: frees the thread to do other work; resumes here after 2 seconds.
await Task.Delay(2000);
```

`Task.Delay` doesn't occupy a thread while waiting — it schedules a timer and the thread is released to do other work in the meantime. This distinction matters enormously in server applications, where blocking a thread pool thread with `Thread.Sleep` can starve the pool under load.

6.5 `Task.Wait()` / `.Result` — and why to avoid them

```
Task<int> task = Task.Run(() => 42);

int result1 = task.Result; // Blocks the calling thread until done.
task.Wait();              // Also blocks.

int result2 = await task;  // Does NOT block the thread – see Chapter 7.
```

`.Result` and `.Wait()` **block the calling thread synchronously**, which defeats much of the purpose of using tasks and — in UI or ASP.NET applications — can cause deadlocks (explained in Chapter 13). Prefer `await` whenever possible.

Chapter 7 — Async and Await

7.1 The problem `async / await` solves

Imagine a method that reads a file, then queries a database, then calls a web API — three operations that each involve **waiting** (for disk, network, etc.), not real CPU work. If you call them synchronously, the calling thread sits idle, doing nothing, for the entire duration of each wait.

`async / await` lets you write code that *looks* sequential and easy to read, but which releases the thread during each wait, so that thread can go do other useful work (serve another web request, respond to another UI event, poll another sensor) instead of blocking.

Key idea: `async / await` is primarily about not wasting threads while waiting on I/O. It is *not* primarily about running things in parallel on multiple cores — that's what Chapter 10 (`Parallel / PLINQ`) is for. This is the single most common point of confusion for people new to C# concurrency.

7.2 Your first `async` method

```
using System;
using System.Net.Http;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        string content = await DownloadPageAsync("https://example.com");
        Console.WriteLine($"Downloaded {content.Length} characters.");
    }

    // The 'async' keyword allows 'await' to be used inside this method.
    // Its return type Task<string> means: "eventually produces a string."
    static async Task<string> DownloadPageAsync(string url)
    {
        using HttpClient client = new HttpClient();

        // 'await' suspends this method (WITHOUT blocking the thread) until
        // the download completes, then resumes with the result.
        string html = await client.GetStringAsync(url);
        return html;
    }
}
```

What actually happens when you hit an `await` :

1. `DownloadPageAsync` calls `client.GetStringAsync(url)` , which starts the network request and immediately returns a `Task<string>` that isn't finished yet.
2. `await` on that task says: "suspend this method here; when the task completes, resume from this exact point — but in the meantime, **give the thread back** so it can do other work."
3. The calling thread is now free. It could go run other queued work, or (in a UI app) keep handling clicks and repainting the window.
4. When the network request finishes, the *continuation* (the rest of the method, after the `await`) runs — usually on a thread-pool thread.

7.3 The rule: `async` methods return `Task`, `Task<T>`, or `void`

```
async Task DoWorkAsync()           // no result – preferred for "fire and forget from
an async caller"
async Task<int> ComputeAsync()      // produces an int
async void OnButtonClick(object s, EventArgs e) // ONLY for event handlers – see 7.6
```

An `async` method's return type is what its *caller* awaits. Inside the method itself, when you `return 42;` from a method declared `async Task<int>`, the compiler wraps it into a completed `Task<int>` for you automatically — you never write `return Task.FromResult(42)` yourself inside an `async` method (that pattern is only for *non-async* methods that still need to return a `Task`).

7.4 `async` all the way down

Once you `await` something inside a method, that method should itself be `async`, and its callers who need the result should `await` it too — this is often summarized as "async all the way up the call stack." Mixing synchronous blocking (`.Result`, `.Wait()`) with asynchronous code is the #1 source of subtle bugs and deadlocks (see 7.7 and Chapter 13).

```
// Good: async flows naturally up the call chain.
static async Task<Reading> ReadSensorAsync(SerialPort port)
{
    byte[] buffer = await ReadBytesAsync(port, 16);
    return ParseReading(buffer);
}

static async Task<byte[]> ReadBytesAsync(SerialPort port, int count)
{
    byte[] buffer = new byte[count];
    await port.BaseStream.ReadAsync(buffer, 0, count);
    return buffer;
}
```

7.5 What `await` is NOT

A common misconception: **`await` does not create a new thread.** It doesn't make code run "in parallel" by itself. All `await` does is:

- Suspend the current method until the awaited `Task` completes.
- Free up the current thread while waiting.

- Resume the method (on a thread-pool thread, typically) once the result is ready.

If the underlying operation is I/O-bound (network, disk, serial port), no thread is actually "busy" during the wait at all — the operating system notifies .NET when the I/O completes, and only *then* does a thread get involved again to run your continuation. If you want genuine multi-core parallel execution of CPU-bound work, you need `Task.Run` (to offload work to a thread-pool thread) or `Parallel/PLINQ` (Chapter 10) — not `await` by itself.

7.6 `async void` — avoid it, except for event handlers

```
// BAD: caller has no Task to await, so exceptions can crash the process
// and the caller cannot know when this finishes.
async void ProcessDataBad()
{
    await Task.Delay(1000);
    throw new Exception("Boom"); // This can crash the whole app – nothing catches it!
}

// GOOD: caller can await it and catch exceptions normally.
async Task ProcessDataGood()
{
    await Task.Delay(1000);
    throw new Exception("Boom"); // Captured in the returned Task; caller's catch works.
}

// The ONE legitimate use of async void: UI event handlers, which must return void
// because that's the delegate signature the framework requires.
private async void OnRefreshButtonClicked(object sender, EventArgs e)
{
    try
    {
        await RefreshDataAsync(); // Always wrap async void bodies in try/catch.
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Refresh failed: {ex.Message}");
    }
}
```

7.7 `ConfigureAwait(false)` — what it does and when to use it

In UI applications (WPF, WinForms) and old-style ASP.NET, there is a **synchronization context** that tries to resume your code on the original thread (e.g., the UI thread) after

an `await`, so you can safely touch UI controls afterward. This has a cost, and in library/non-UI code you usually don't need it.

```
public async Task<string> LoadConfigAsync(string path)
{
    // Tells the runtime: "I don't need to resume on the original context;
    // any thread-pool thread is fine." This can improve performance and,
    // importantly, avoid deadlocks in certain mixed sync/async scenarios.
    string text = await File.ReadAllTextAsync(path).ConfigureAwait(false);
    return text.Trim();
}
```

Rule of thumb: use `ConfigureAwait(false)` in library/backend code that never touches UI elements. Do *not* use it in code that needs to get back onto the UI thread afterward (e.g., right before updating a WPF control). Modern ASP.NET Core has no synchronization context at all, so `ConfigureAwait(false)` matters less there, but many teams still use it consistently in library code out of habit and clarity.

7.8 A complete, realistic example: reading from a device asynchronously

```
using System;
using System.IO.Ports;
using System.Threading.Tasks;

class SensorReader
{
    private readonly SerialPort _port;

    public SensorReader(string portName)
    {
        _port = new SerialPort(portName, baudRate: 115200);
        _port.Open();
    }

    // Reads one line of text from the sensor without blocking the calling thread.
    public async Task<string> ReadLineAsync()
    {
        // SerialPort doesn't have a native async line-read, so we offload the
        // blocking call to a background thread with Task.Run, then await it.
        return await Task.Run(() => _port.ReadLine());
    }

    public async Task<double> ReadTemperatureAsync()
    {
        string line = await ReadLineAsync();
        if (double.TryParse(line, out double temperature))
        {
            return temperature;
        }
        throw new FormatException($"Unexpected sensor output: '{line}'");
    }
}

class Program
{
    static async Task Main()
    {
        var reader = new SensorReader("COM3");

        for (int i = 0; i < 5; i++)
        {
            double temp = await reader.ReadTemperatureAsync();
            Console.WriteLine($"Reading {i}: {temp:F1} degrees");
        }
    }
}
```

Notice how `Main` reads almost like fully synchronous code — that readability, combined with non-blocking behavior underneath, is the whole point of `async / await`.

Chapter 8 — Combining Tasks: Continuations, WhenAll, WhenAny

8.1 `Task.WhenAll` — run several tasks concurrently and wait for all of them

```
using System;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        Task<int> taskA = FetchValueAsync("A", 1000);
        Task<int> taskB = FetchValueAsync("B", 1500);
        Task<int> taskC = FetchValueAsync("C", 800);

        // All three run CONCURRENTLY (they were already started above).
        // WhenAll waits for all of them and returns their results as an array.
        int[] results = await Task.WhenAll(taskA, taskB, taskC);

        Console.WriteLine($"Sum: {results[0] + results[1] + results[2]}");
        // Total time taken is close to the SLOWEST task (1500 ms), not the SUM (3300
ms).
    }

    static async Task<int> FetchValueAsync(string name, int delayMs)
    {
        await Task.Delay(delayMs);
        Console.WriteLine($"{name} finished after {delayMs} ms");
        return delayMs;
    }
}
```

Important: the three tasks above were already *started* (running) the moment `FetchValueAsync` was called, before `Task.WhenAll` was even reached. `Task.WhenAll` doesn't start anything — it just waits. This is a common beginner mistake:

```
// WRONG: this runs sequentially, one after another, because each is
// awaited (and finishes) before the next one is even started.
int a = await FetchValueAsync("A", 1000);
int b = await FetchValueAsync("B", 1500);
int c = await FetchValueAsync("C", 800);
// Total time: 1000 + 1500 + 800 = 3300 ms

// RIGHT: start all three first (don't await yet), THEN await together.
Task<int> taskA = FetchValueAsync("A", 1000);
Task<int> taskB = FetchValueAsync("B", 1500);
Task<int> taskC = FetchValueAsync("C", 800);
int[] results = await Task.WhenAll(taskA, taskB, taskC);
// Total time: ~1500 ms (the slowest one)
```

8.2 Task.WhenAny — react to whichever finishes first

```
Task<string> primary = FetchFromPrimaryServerAsync();
Task<string> backup = FetchFromBackupServerAsync();

Task<string> firstDone = await Task.WhenAny(primary, backup);
string result = await firstDone; // Get the result (and re-throw if it faulted).

Console.WriteLine($"Fastest response: {result}");
```

`WhenAny` is useful for timeouts, racing redundant sources, or "whichever sensor responds first wins."

A timeout pattern using `WhenAny`

```
static async Task<string> ReadWithTimeoutAsync(Task<string> operation, TimeSpan timeout)
{
    Task delay = Task.Delay(timeout);
    Task finished = await Task.WhenAny(operation, delay);

    if (finished == delay)
    {
        throw new TimeoutException("Operation timed out.");
    }

    return await operation; // Already finished – returns immediately.
}
```

8.3 Continuations with `.ContinueWith` (and why `await` is usually better)

Before `async / await` existed (pre-C# 5), you chained work using `.ContinueWith`:

```
Task.Run(() => ComputeSomething())
    .ContinueWith(prevTask =>
    {
        Console.WriteLine($"Result was {prevTask.Result}");
    });
```

This works, but is harder to read, easy to get wrong around exception handling and thread context, and has mostly been superseded by `await`. Prefer `await` in new code; you'll mainly encounter `.ContinueWith` in older codebases.

Chapter 9 — Cancellation

9.1 The problem

Long-running operations (a download, a loop over millions of items, a background polling loop) sometimes need to be stopped early — the user closed the window, a timeout expired, or another part of the app decided the result is no longer needed. C# has a standard, cooperative mechanism for this: `CancellationToken`.

"Cooperative" is the key word — cancellation in .NET does not forcibly kill a thread. The running code must periodically check "was I asked to cancel?" and voluntarily stop. There is no safe way to force-stop an arbitrary thread from the outside (and `Thread.Abort` is obsolete and dangerous — it can leave shared state half-updated).

9.2 Basic usage

```
using System;
using System.Threading;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        using var cts = new CancellationTokenSource();

        // Cancel automatically after 3 seconds.
        cts.CancelAfter(TimeSpan.FromSeconds(3));

        try
        {
            await CountForeverAsync(cts.Token);
        }
        catch (OperationCanceledException)
        {
            Console.WriteLine("Counting was cancelled.");
        }
    }

    static async Task CountForeverAsync(CancellationToken token)
    {
        int i = 0;
        while (true)
        {
            // Throws OperationCanceledException if cancellation was requested.
            token.ThrowIfCancellationRequested();

            Console.WriteLine(i++);
            await Task.Delay(500, token); // Many async APIs accept a token directly.
        }
    }
}
```


9.3 Cancelling from another part of the program

```
CancellationTokenSource cts = new CancellationTokenSource();

// Somewhere else, e.g. a "Stop" button handler:
void OnStopButtonClicked()
{
    cts.Cancel(); // Signals every operation using cts.Token.
}

// The worker checks the token periodically:
async Task ProcessItemsAsync(IEnumerable<int> items, CancellationToken token)
{
    foreach (int item in items)
    {
        token.ThrowIfCancellationRequested(); // Check before doing more work.
        await ProcessOneAsync(item, token);
    }
}
```

9.4 Passing tokens through, not storing them

A `CancellationToken` should be passed as a parameter down the call chain (as shown above), not stored as a field, because a single token is meant to represent "this one operation," not "cancellation forever." Almost every modern async API in .NET (`HttpClient`, `Task.Delay`, `Stream.ReadAsync`, database drivers, etc.) accepts an optional trailing `CancellationToken` parameter — always pass it through when you have one, so cancellation actually propagates all the way down.

9.5 Timeouts with `CancellationTokenSource`

```
using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(5));
try
{
    string result = await httpClient.GetStringAsync(url, cts.Token);
}
catch (OperationCanceledException)
{
    Console.WriteLine("Request timed out after 5 seconds.");
}
```

Chapter 10 — Parallel Programming: `Parallel.For` and PLINQ

10.1 When to reach for this chapter

Everything up to now has mostly been about **not wasting a thread while waiting** (async/await) or **coordinating access to shared data** (locks). This chapter is different: it's about **splitting CPU-bound work across multiple cores to finish faster**. Use it when you have a batch of independent calculations to perform, and your machine has multiple cores to throw at them.

10.2 `Parallel.For` and `Parallel.ForEach`

```
using System;
using System.Threading.Tasks;

class Program
{
    static void Main()
    {
        double[] input = new double[10_000_000];
        double[] output = new double[input.Length];

        // Fill input with some data...
        for (int i = 0; i < input.Length; i++) input[i] = i;

        // Sequential version (single core):
        // for (int i = 0; i < input.Length; i++) { output[i] =
        ExpensiveCalc(input[i]); }

        // Parallel version: the runtime automatically splits the range across
        // multiple threads / cores.
        Parallel.For(0, input.Length, i =>
        {
            output[i] = ExpensiveCalc(input[i]);
        });

        Console.WriteLine("Done.");
    }

    static double ExpensiveCalc(double x) => Math.Sqrt(x) * Math.Sin(x);
}
```

```
// Parallel.ForEach works the same way over any IEnumerable<T>.
List<string> filePaths = GetAllFilePaths();

Parallel.ForEach(filePaths, path =>
{
    ProcessFile(path); // Each file processed independently, on any available core.
});
```

Important caveat: the body you pass to `Parallel.For/Parallel.ForEach` runs on *multiple threads simultaneously*. If that body reads or writes any shared, mutable state (not just its own local variables), you're back to the race-condition problem from Chapter 3 and need locks or thread-safe accumulation (see 10.4).

10.3 Controlling the degree of parallelism

```
var options = new ParallelOptions
{
    MaxDegreeOfParallelism = 4 // Never use more than 4 threads, even if more cores exist.
};

Parallel.ForEach(filePaths, options, path =>
{
    ProcessFile(path);
});
```

This matters when you want to leave CPU headroom for other work (e.g., on an embedded device where you don't want to saturate every core).

10.4 Safely accumulating results: thread-local state

```
using System.Threading;

object totalLock = new object();
long total = 0;

Parallel.For<long>(
    0, 10_000_000,
    () => 0L, // Initial value for each thread's LOCAL accumulator.
    (i, state, localSum) => localSum + i, // Runs per-iteration, updates local sum only.
    localSum =>
    {
        // Runs once per thread when it's done – merge into the shared total safely.
        lock (totalLock)
        {
            total += localSum;
        }
    }
);

Console.WriteLine($"Total: {total}");
```

This pattern avoids taking a lock on *every single iteration* (which would be slow); instead, each thread accumulates locally and only synchronizes once at the end.

10.5 PLINQ — parallel LINQ

If you're already comfortable with LINQ, PLINQ lets you parallelize a query with a single method call:

```

using System.Linq;

int[] numbers = Enumerable.Range(0, 10_000_000).ToArray();

// Sequential LINQ:
var sequentialResult = numbers.Where(IsPrime).Count();

// Parallel LINQ – same query, but split across cores automatically:
var parallelResult = numbers.AsParallel()
                             .Where(IsPrime)
                             .Count();

static bool IsPrime(int n)
{
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++)
    {
        if (n % i == 0) return false;
    }
    return true;
}

```

`AsParallel()` is a single call that turns a LINQ pipeline into a parallel one. It shines on CPU-bound, independent, per-element work over large collections; it's usually *not* a win for small collections (the overhead of splitting work across threads can exceed the time saved) or for I/O-bound work (use `async / await` for that instead).

10.6 `Parallel` vs. `Task.Run` in a loop — don't do it yourself

A common beginner mistake is manually spinning up one `Task` per item instead of using `Parallel`:

```

// Don't do this – creates one Task PER ITEM, with no batching or throttling.
List<Task> tasks = new List<Task>();
foreach (var item in hugeList)
{
    tasks.Add(Task.Run(() => Process(item)));
}
await Task.WhenAll(tasks);

// Prefer Parallel.ForEach – it batches work intelligently across a bounded
// number of threads instead of creating thousands of Task objects at once.
Parallel.ForEach(hugeList, item => Process(item));

```

Chapter 11 — Concurrent Collections

11.1 The problem

Regular collections (`List<T>`, `Dictionary<TKey, TValue>`, `Queue<T>`) are **not thread-safe**. If multiple threads add to a `List<T>` at the same time without a lock, you can get corrupted internal state, lost items, or exceptions.

You *can* wrap any collection in a `lock` (as shown in Chapter 4), but the `System.Collections.Concurrent` namespace provides collections that are already designed and optimized for safe multithreaded access, usually with better performance than a hand-rolled lock around a regular collection.

11.2 `ConcurrentDictionary<TKey, TValue>`

```
using System.Collections.Concurrent;
using System.Threading.Tasks;

var deviceStatus = new ConcurrentDictionary<string, string>();

Parallel.ForEach(new[] { "dev-1", "dev-2", "dev-3" }, deviceId =>
{
    // Safe to call from many threads at once – no external lock needed.
    deviceStatus[deviceId] = "online";
});

// Atomic "add or update" – useful for counters keyed by something.
var requestCounts = new ConcurrentDictionary<string, int>();
requestCounts.AddOrUpdate(
    "dev-1",
    addValueFactory: _ => 1,                // If key doesn't exist yet, start at 1.
    updateValueFactory: (_, existing) => existing + 1); // If it exists, increment it.
```

11.3 `ConcurrentQueue<T>` and `ConcurrentBag<T>`

```
using System.Collections.Concurrent;

var queue = new ConcurrentQueue<int>();

queue.Enqueue(1);
queue.Enqueue(2);

if (queue.TryDequeue(out int value))
{
    Console.WriteLine($"Dequeued: {value}");
}
```

`ConcurrentBag<T>` is an unordered thread-safe collection, best when you don't care about item order and want the fastest possible concurrent adds (e.g., collecting results from parallel workers).

11.4 `BlockingCollection<T>` — the classic producer-consumer building block

`BlockingCollection<T>` wraps a concurrent collection and adds the ability to **block** a consumer thread when the collection is empty (instead of returning immediately), and to signal "no more items are coming" via `CompleteAdding()`. This is the standard tool for the producer-consumer pattern, used heavily in Chapter 12.

```
using System.Collections.Concurrent;

var buffer = new BlockingCollection<int>(boundedCapacity: 100);

// Producer:
buffer.Add(42);
buffer.CompleteAdding(); // Signals "no more items will be added."

// Consumer:
foreach (int item in buffer.GetConsumingEnumerable())
{
    Console.WriteLine(item); // Blocks here if buffer is empty, until an item
                             // arrives OR CompleteAdding() has been called.
}
```

Chapter 12 — Producer-Consumer Pattern (Putting It All Together)

This chapter combines almost every concept from the book into one realistic example: a system that reads sensor readings from a serial device on a background thread (producer), while one or more worker tasks process and forward them (consumers) — with cancellation, bounded buffering, and safe shared counters.


```

using System;
using System.Collections.Concurrent;
using System.Threading;
using System.Threading.Tasks;

class SensorReading
{
    public int DeviceId { get; init; }
    public double Value { get; init; }
    public DateTime Timestamp { get; init; }
}

class SensorPipeline
{
    // Bounded buffer: if it fills up (capacity 500), the producer will block
    // on Add() until a consumer makes room – this naturally prevents runaway
    // memory growth if consumers fall behind.
    private readonly BlockingCollection<SensorReading> _buffer =
        new BlockingCollection<SensorReading>(boundedCapacity: 500);

    private int _processedCount = 0; // Read/written by multiple consumer tasks.

    // --- PRODUCER: runs on its own dedicated thread (Chapter 2), because it
    // continuously blocks on hardware I/O and never really "finishes." ---
    public void StartProducer(CancellationToken token)
    {
        Thread producerThread = new Thread(() => ProduceReadings(token))
        {
            IsBackground = true,
            Name = "SensorProducer"
        };
        producerThread.Start();
    }

    private void ProduceReadings(CancellationToken token)
    {
        var random = new Random();
        try
        {
            while (!token.IsCancellationRequested)
            {
                var reading = new SensorReading
                {
                    DeviceId = random.Next(1, 4),
                    Value = random.NextDouble() * 100,
                    Timestamp = DateTime.UtcNow
                };

                // Blocks (this thread only) if the buffer is full.
                _buffer.Add(reading, token);

                Thread.Sleep(10); // Simulate the pace of incoming hardware data.
            }
        }
    }
}

```

```

    }
    catch (OperationCanceledException)
    {
        // Expected on shutdown.
    }
    finally
    {
        _buffer.CompleteAdding();
    }
}

// --- CONSUMERS: several Tasks pulling from the same buffer concurrently. ---
public async Task RunConsumersAsync(int consumerCount, CancellationToken token)
{
    Task[] consumers = new Task[consumerCount];
    for (int i = 0; i < consumerCount; i++)
    {
        int consumerId = i;
        consumers[i] = Task.Run(() => ConsumeReadings(consumerId, token), token);
    }

    await Task.WhenAll(consumers);
    Console.WriteLine($"All consumers finished. Total processed: {_processedCount}")
;
}

private void ConsumeReadings(int consumerId, CancellationToken token)
{
    // GetConsumingEnumerable blocks between items and exits cleanly once
    // CompleteAdding() has been called AND the buffer is empty.
    foreach (SensorReading reading in _buffer.GetConsumingEnumerable(token))
    {
        ProcessReading(consumerId, reading);

        // Interlocked.Increment avoids a race condition on the shared counter
        // without needing a full lock (Chapter 4.4).
        Interlocked.Increment(ref _processedCount);
    }
}

private void ProcessReading(int consumerId, SensorReading reading)
{
    Console.WriteLine(
        $"[Consumer {consumerId}] Device {reading.DeviceId}: " +
        $"{reading.Value:F2} at {reading.Timestamp:HH:mm:ss.fff}");
}
}

class Program
{
    static async Task Main()
    {
        var pipeline = new SensorPipeline();
        using var cts = new CancellationTokenSource();
    }
}

```

```

        pipeline.StartProducer(cts.Token);
        Task consumerTask = pipeline.RunConsumersAsync(consumerCount: 3, cts.Token);

        Console.WriteLine("Pipeline running. Press Enter to stop.");
        Console.ReadLine();

        cts.Cancel();           // Signal the producer to stop and consumers to unwind.
        await consumerTask;     // Wait for everything to shut down cleanly.
    }
}

```

What this example demonstrates, and where:

Concept	Where in the code
Dedicated Thread for continuous hardware I/O	StartProducer
Task.Run for a pool of concurrent workers	RunConsumersAsync
Bounded buffer / backpressure	BlockingCollection(boundedCapacity: 500)
Cancellation	CancellationToken threaded through every layer
Lock-free shared counter	Interlocked.Increment(ref _processedCount)
async / await for waiting on multiple workers	await Task.WhenAll(consumers)

Chapter 13 — Common Pitfalls and Best Practices

13.1 Deadlocks from mixing `.Result / .Wait()` with `async`

This is the single most common real-world async bug in UI and old-style ASP.NET apps.

```
// In a WPF/WinForms button click handler:
void OnButtonClick(object sender, EventArgs e)
{
    // DEADLOCK RISK: blocks the UI thread waiting for a task that, once it
    // resumes (see Chapter 7.7), tries to get BACK ONTO the UI thread – but
    // the UI thread is stuck right here, waiting. Neither can proceed.
    var result = LoadDataAsync().Result;
}

async Task<string> LoadDataAsync()
{
    await Task.Delay(1000); // By default, tries to resume on the original
                          // (UI) synchronization context.
    return "done";
}
```

Fixes: - Make the caller `async` too, and `await` instead of using `.Result`: the standard, correct fix. - Or use `ConfigureAwait(false)` inside `LoadDataAsync` so its continuation doesn't try to resume on the UI thread (a workaround, not a substitute for fix #1 where possible).

13.2 `async void` swallowing exceptions

Covered in 7.6 — an exception thrown inside an `async void` method cannot be caught by the caller with a normal `try/catch`, because there's no `Task` to observe. Only use `async void` for event handlers, and always wrap the body in `try/catch`.

13.3 Forgetting to `await` (fire-and-forget by accident)

```
// BUG: SaveDataAsync() is called but never awaited. The method returns
// immediately; SaveDataAsync might still be running (or might throw an
// exception nobody ever observes) after this method has already returned.
void SaveData()
{
    SaveDataAsync(); // Compiler warning: "because this call is not awaited..."
}

async Task SaveDataAsync() { /* ... */ }
```

Always heed the compiler warning `CS4014` ("this call is not awaited"). If you genuinely mean to fire-and-forget, be explicit about it and handle exceptions:

```
_ = SaveDataAsync().ContinueWith(t =>
{
    if (t.Exception != null) LogError(t.Exception);
}, TaskContinuationOptions.OnlyOnFaulted);
```

13.4 Capturing loop variables incorrectly

```
// Modern C# (5+) handles 'foreach' loop variables correctly per iteration.
// This prints 0, 1, 2 (in some order) as expected:
foreach (int i in new[] { 0, 1, 2 })
{
    Task.Run(() => Console.WriteLine(i));
}

// A classic 'for' loop with a shared counter variable is more subtle – always
// copy the loop variable into a fresh local before capturing it in a closure,
// to make the per-iteration snapshot explicit and avoid surprises:
for (int i = 0; i < 3; i++)
{
    int captured = i;
    Task.Run(() => Console.WriteLine(captured));
}
```

13.5 Locking on the wrong object

```
// BAD: locking on 'this' or on a string can cause unrelated code elsewhere
// in the app (that happens to lock on the same object) to interfere.
lock (this) { /* ... */ }
lock ("some-string") { /* ... */ } // Strings can be interned and shared!

// GOOD: a private, dedicated object that nothing else can reach.
private readonly object _sync = new object();
lock (_sync) { /* ... */ }
```

13.6 Using `Parallel.For` / PLINQ for I/O-bound work

`Parallel.For` and PLINQ are for CPU-bound work. Using them to run many I/O-bound operations (e.g., HTTP calls) ties up thread-pool threads waiting on I/O for no benefit — use `Task.WhenAll` with genuinely async operations instead.

```
// BAD: ties up a thread-pool thread per blocking HTTP call.
Parallel.ForEach(urls, url =>
{
    string content = httpClient.GetStringAsync(url).Result; // blocking!
});

// GOOD: async all the way, no threads wasted waiting on the network.
var tasks = urls.Select(url => httpClient.GetStringAsync(url));
string[] results = await Task.WhenAll(tasks);
```

13.7 Over-using locks / excessive contention

Locking too broadly (e.g., one giant lock around your entire data layer) serializes work that could otherwise run concurrently, erasing the benefit of multithreading. Lock the smallest, most specific piece of shared state you can, for the shortest time possible, and prefer purpose-built concurrent collections (Chapter 11) over hand-rolled locking wherever they fit.

13.8 Quick checklist before writing concurrent code

- Is this operation CPU-bound (use `Task.Run` / `Parallel` / PLINQ) or I/O-bound (use `async` / `await` natively — no `Task.Run` needed for genuinely async APIs)?
- Is any state shared and mutable across threads? If so, protect it (`lock` , `Interlocked` , or a concurrent collection).

- Am I `await`-ing everything I start, or did I forget one (CS4014)?
- Does this method need to support cancellation? If it might run for more than a fraction of a second, probably yes.
- Am I mixing blocking calls (`.Result` , `.Wait()` , `Thread.Sleep`) into otherwise-async code? Don't.

Chapter 14 — Cheat Sheet / Quick Reference

Choosing your tool

I need to...	Use
Run one background CPU-bound calculation	<code>Task.Run(...)</code>
Call an I/O API (file, network, DB, serial) without blocking	<code>async / await</code> with the API's native <code>...Async</code> method
Run a long-lived, dedicated background worker	<code>Thread</code> with <code>IsBackground = true</code>
Split a big loop of independent work across cores	<code>Parallel.For</code> / <code>Parallel.ForEach</code>
Parallelize a LINQ query	<code>.AsParallel()</code>
Wait for several tasks to all finish	<code>await Task.WhenAll(...)</code>
React to whichever of several tasks finishes first	<code>await Task.WhenAny(...)</code>
Protect a critical section from concurrent access	<code>lock (privateObj) { ... }</code>
Protect a critical section that itself needs to <code>await</code>	<code>SemaphoreSlim</code> with <code>WaitAsync()</code>
Increment/update a single shared value safely, fast	<code>Interlocked.Increment</code> / <code>.Add</code> / <code>.CompareExchange</code>
Allow many readers, one writer	<code>ReaderWriterLockSlim</code>
Share a collection safely across threads	<code>ConcurrentDictionary</code> , <code>ConcurrentQueue</code> , <code>ConcurrentBag</code>
Build a producer/consumer pipeline	<code>BlockingCollection<T></code>
Support stopping an operation early	<code>CancellationToken</code> / <code>CancellationTokenSource</code>

Syntax quick reference

```
// Starting work
Task t = Task.Run(() => DoWork());
Task<int> t2 = Task.Run(() => Compute());

// Waiting (non-blocking, inside an async method)
await t;
int result = await t2;

// Waiting for several
int[] results = await Task.WhenAll(t2, t2, t2);
Task first = await Task.WhenAny(t, t2);

// Cancellation
using var cts = new CancellationTokenSource();
cts.CancelAfter(TimeSpan.FromSeconds(5));
await SomeAsyncMethod(cts.Token);

// Locking
private readonly object _sync = new object();
lock (_sync) { /* short, critical section */ }

// Atomic counter
Interlocked.Increment(ref counter);

// Parallel loop
Parallel.ForEach(items, item => Process(item));

// Async method declaration patterns
async Task DoWorkAsync() { await Task.Delay(100); }
async Task<int> ComputeAsync() { await Task.Delay(100); return 42; }
```

The three questions to ask before touching threads

1. **Is the work CPU-bound or I/O-bound?** CPU-bound → `Task.Run` / `Parallel`. I/O-bound → `async` / `await` with a native async API (no extra thread needed).
2. **Is any state shared and mutable?** If yes → lock it, make it atomic (`Interlocked`), or use a concurrent collection.
3. **Does it need to be cancellable or time out?** If yes → thread a `CancellationToken` through every layer.

End of book.